Score-based Data Assimilation

FRANÇOIS ROZET AND GILLES LOUPPE

Presented by Paul de Nailly, Ronan Fablet

AI4PEX

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Content

Data Assimilation

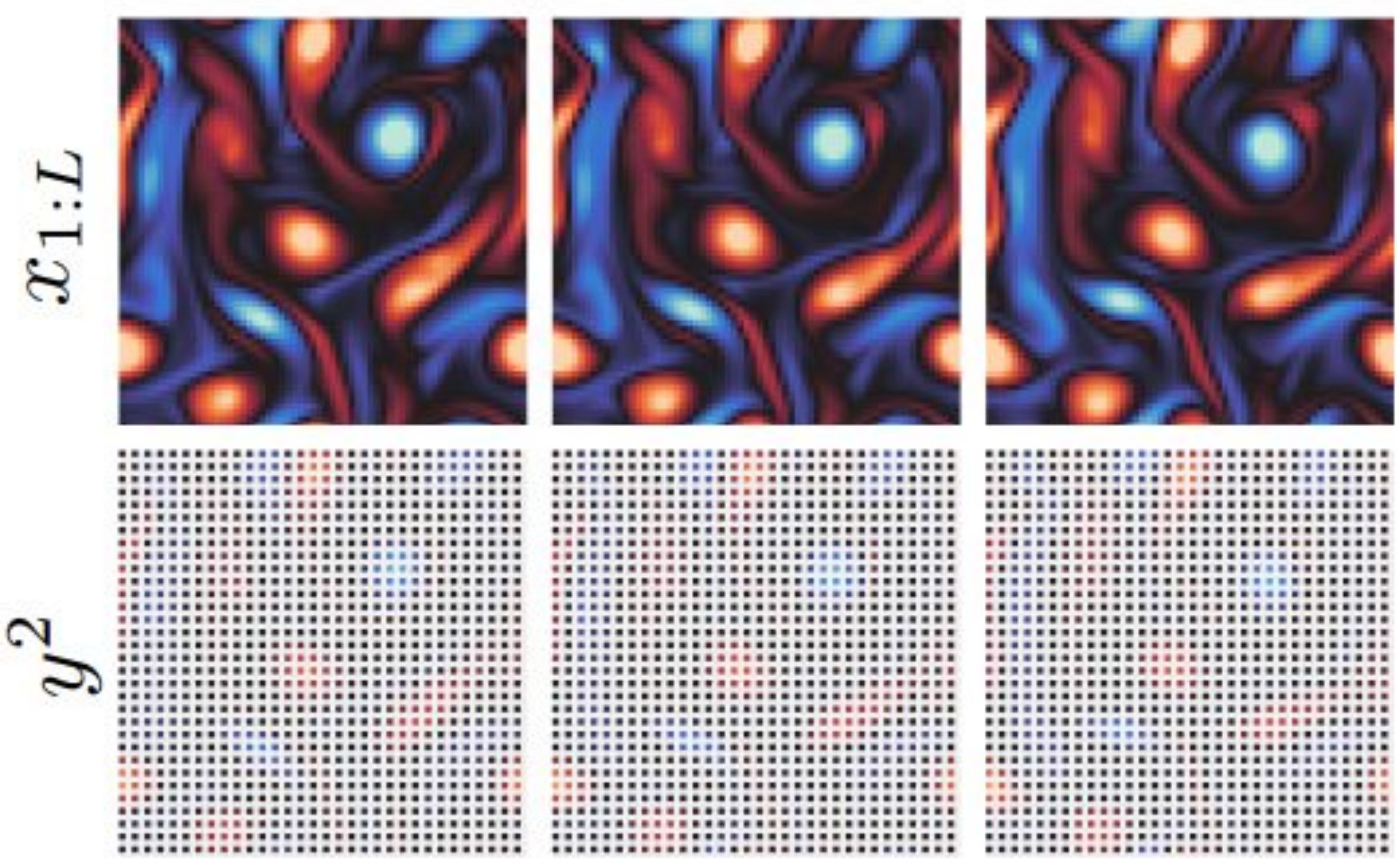
Data assimilation

Goal:

Estimate the **state** of a system as it evolves over time using imperfect sources of information.

The **observations** are:

- **sparse** in both space and time, leading to coverage gaps
- **noisy** due to measurement errors and instrument uncertainty



Data assimilation

Goal:

Estimate the **state** of a system as it evolves over time using imperfect sources of information.

The **observations** are:

- **sparse** in both space and time, leading to coverage gaps
- **noisy** due to measurement errors and instrument uncertainty

Notations:

$$x_{1:L} = (x_1, x_2, \ldots, x_L) \in \mathbb{R}^{L \times D}$$

a **trajectory** of D states in a L steps discrete-time

$$p(x_{i+1} \mid x_i)$$

the **transition** dynamics from state xi to state xi+1

$$y \in \mathbb{R}^M$$

the **observation** of the state trajectory

Data assimilation

State space formulation:

$$\begin{cases} x_1 \sim \mathcal{N}(x_b, B) & \Rightarrow p(x_1) & \text{(Initial State / Background)} \\ x_{i+1} = \mathcal{M}(x_i) + \eta_i,\ \eta_i \sim \mathcal{N}(0, Q) & \Rightarrow p(x_{i+1}|x_i) & \text{(Transition Model / Physics)} \\ y_i = \mathcal{A}(x_i) + \epsilon_i,\ \epsilon_i \sim \mathcal{N}(0, R) & \Rightarrow p(y_i|x_i) & \text{(Likelihood / Observation)} \end{cases}$$

Bayesian framework:

$$p(x_{1:L}|y) = \frac{p(y|x_{1:L})}{p(y)} p(x_1) \prod_{i=1}^{L-1} p(x_{i+1}|x_i)$$

Score based generative models

Score based generative models

What for? Image, video or audio generation. Good for data assimilation problems?

<u>Unconditional sampling</u>

I - Forward process (from Data to Noise)

Progressively perturbs a physical trajectory x(0) into pure noise x(1) over continuous time

$t \in [0, 1]$ via a Stochastic Differential Equation (SDE):

$$dx(t) = f(t)x(t)dt + g(t)dw(t)$$

Drift (scales data) **Diffusion (adds noise)**

Score based generative models

What for? Image, video or audio generation. Good for data assimilation problems?

Unconditional sampling

I - Forward process (from Data to Noise)

Progressively perturbs a physical trajectory x(0) into pure noise x(1) over continuous time

t ∈ [0, 1] via a Stochastic Differential Equation (SDE):

$$dx(t) = \underbrace{f(t)x(t)dt}_{\text{Drift (scales data)}} + \underbrace{g(t)dw(t)}_{\text{Diffusion (adds noise)}}$$

II - Reverse Process (from Noise to Data)

Generate valid trajectories from noise. It requires the **marginal score vector field**:

$$dx(t) = \left[f(t)x(t) - g(t)^2\nabla_{x(t)} \log p(x(t))\right] dt + g(t)d\bar{w}(t)$$

Score based generative models

Problem

True score $\nabla_{x(t)} \log p(x(t))$ uncomputable because the data density p(x(t)) is unknown and high-dimensional

Solution: Denoising score matching

- Train a neural network **sϕ** to minimize the **divergence** between the **network output** and the **known score** of the perturbation kernel:

$$\arg\min_{\phi} \mathbb{E}_{p(x)p(t)p(x(t)|x)} \left[\sigma(t)^2 \| s_\phi(x(t), t) - \nabla_{x(t)} \log p(x(t)|x) \|_2^2\right]$$

GAUSSIAN DISTRIBUTION

- Error minimized against the conditional score matches the marginal score, so then:

$$\boxed{s_\phi(x(t), t) \approx \nabla_{x(t)} \log p(x(t))}$$

Score based generative models

Conditional sampling

- We want to **condition** the diffusion process on the **observation y**
- We could directly learn a conditional score **sϕ(x(t), t | y)**

Problem

Score would need to be **re-trained** each time the observation process p(y|x) changes

Solution

We observe from the Bayes formula

$$\nabla_{x(t)} \log p(x(t)|y) = \nabla_{x(t)} \log p(x(t)) + \nabla_{x(t)} \log p(y|x(t))$$

we need to compute $\nabla_{x(t)} \log p(y \mid x(t))$ analytically

Score based generative models

Conditional sampling

Assumption:

$$p(y \mid x) = \mathcal{N}(y \mid \mathcal{A}(x), \Sigma_y)$$

Chung et al. (2) propose the approximation:

$$\boxed{p(y \mid x(t)) = \int p(y \mid x)\, p(x \mid x(t))\, \mathrm{d}x \approx \mathcal{N}\left(y \mid \mathcal{A}(\hat{x}(x(t))), \Sigma_y\right)}$$

where the mean $\hat{x}(x(t)) = \mathbb{E}_{p(x|x(t))}[x]$ is given by Tweedie's formula:

$$\begin{aligned} \mathbb{E}_{p(x|x(t))}[x] &= \frac{x(t) + \sigma(t)^2\, \nabla_{x(t)} \log p(x(t))}{\mu(t)} \\ &\approx \frac{x(t) + \sigma(t)^2\, \boxed{s_\phi(x(t), t)}}{\mu(t)}. \end{aligned}$$

there is no need to train any other network than sϕ(x(t), t)

Score based DA

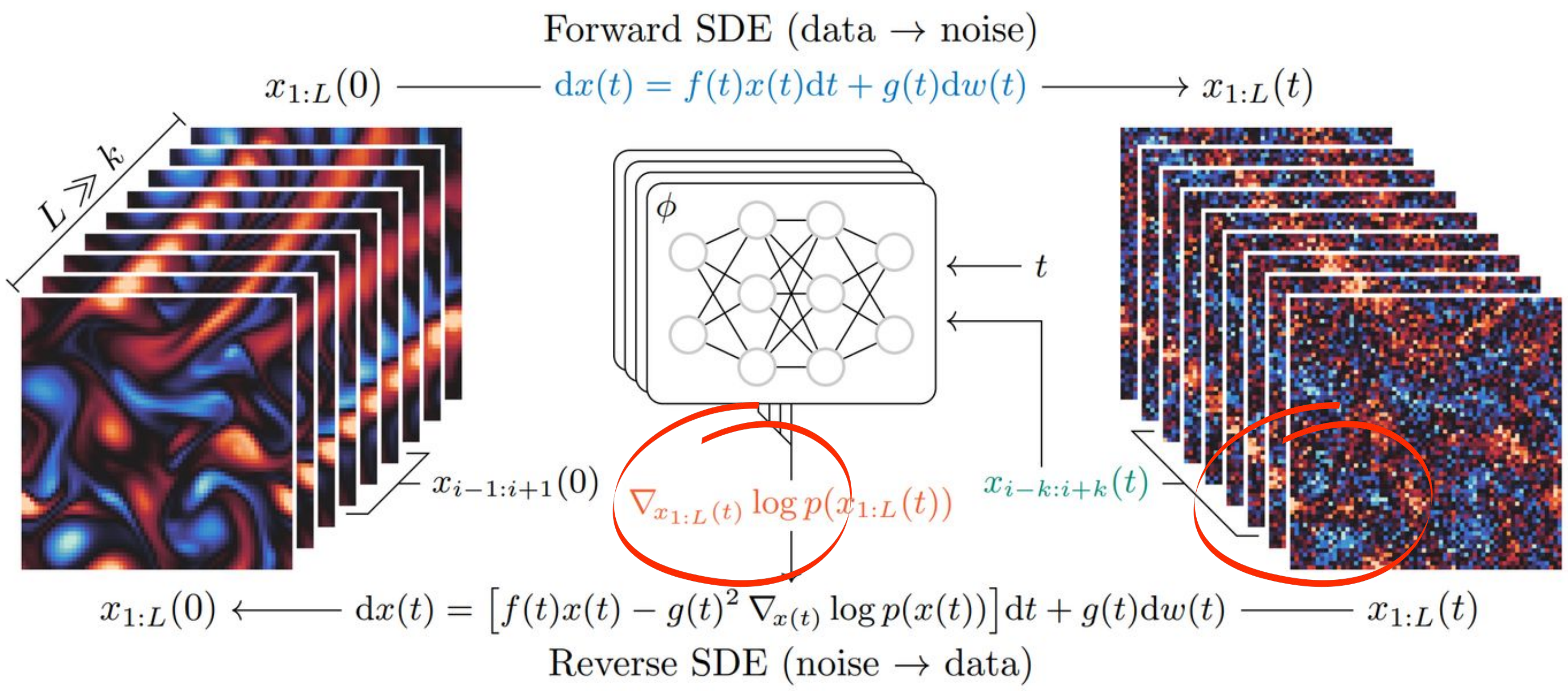
General problem

"How can we use score based generative models for data assimilation?"

Problem

Too large amount of data for training, which have in addition to be simulated from noisy observations

Solution



*Score approximated by combining the outputs of a score network over **subsegments** of x1:L(t)*

Markov blanket

How to handle large-scale states?

Principle

In a time series, a state x_i depends only on its direct neighbors

$$p(x_i \mid x_{\neq i}) = p(x_i \mid x_{b_i})$$

(The state depends on its “blanket” x_{b_i} not on the rest)

Proposition: Pseudo-Blanket

- Set a window with a parameter **k** (found experimentally).
- Windows : $\bar{b}_i = i - k; i + k$

$$\nabla_{x_i(t)} \log p(x_{1:L}(t)) \approx \nabla_{x_i(t)} \log p(x_i(t), x_{\bar{b}_i}(t))$$

Final advantage: Training on short segments → Massive gains in time and computational cost!

Results - Lorentz 1963

Objective : retrieving the conditionnal distribution $q(x_{1:L} \mid y)$ for a stochastic dynamic model

1st step : Building a simple stochastic dynamic model

The state $x = (a, b, c) \in \mathbb{R}^3$ of the Lorenz system evolves according to a system of ordinary differential equations

$$\begin{aligned} \dot{a} &= \sigma(b - a) \\ \dot{b} &= a(\rho - c) - b \\ \dot{c} &= ab - \beta c \end{aligned} \tag{18}$$

$x_{i+1} = \mathcal{M}(x_i) + \eta$, where $\mathcal{M} : \mathbb{R}^3 \mapsto \mathbb{R}^3$ is the integration of the differential equations

Why this model → we have an algorithm (BPF) to sample the groundtruth $p(x_{1:L} \mid y)$

Conditional diffusion

How to stabilize conditional diffusion if noise is high?

Principle

Propose a better approximation of the perturbed likelihood $p(y|x(t))$ to stabilize the inference.

Proposition

Adding a new covariance term to better handle uncertainty on x(t) computation.

From…

$$p(y \mid x(t)) \approx \mathcal{N}\left(y \mid \mathcal{A}(\hat{x}(x(t))), \Sigma_y\right)$$

…to

$$p(y \mid x(t)) \approx \mathcal{N}\left(y \mid \mathcal{A}(\hat{x}(x(t))), \Sigma_y + \boxed{\frac{\sigma(t)^2}{\mu(t)^2} A \Gamma A^T}\right)$$

Predictor Corrector sampling

How to solve reverse SDE numerically without accumulating too much errors?

Principle

Scores used are approximations, so errors can accumulate quickly during generation

Proposition : Predictor Corrector sampling

Predictor

- Move the reverse SDE, remove noise
- uses a numerical scheme called
  "Exponential Integrator" (Zhang et al., 2023) :

$$x(t-\Delta t) \leftarrow \frac{\mu(t-\Delta t)}{\mu(t)} x(t) + \left(\frac{\mu(t-\Delta t)}{\mu(t)} - \frac{\sigma(t-\Delta t)}{\sigma(t)}\right) \sigma(t)^2 \, s_\phi(x(t), t)$$

- Issue : the score approximation implies numerical errors

Corrector

- Langevin Monte Carlo (LMC) scheme with C steps:

$$x(t) \leftarrow x(t) + \delta \, s_\phi(x(t), t) + \sqrt{2\delta}\,\epsilon$$

- Effect : push the sample towards a high probability region

Results on Lorentz 1963 experiment

Objective

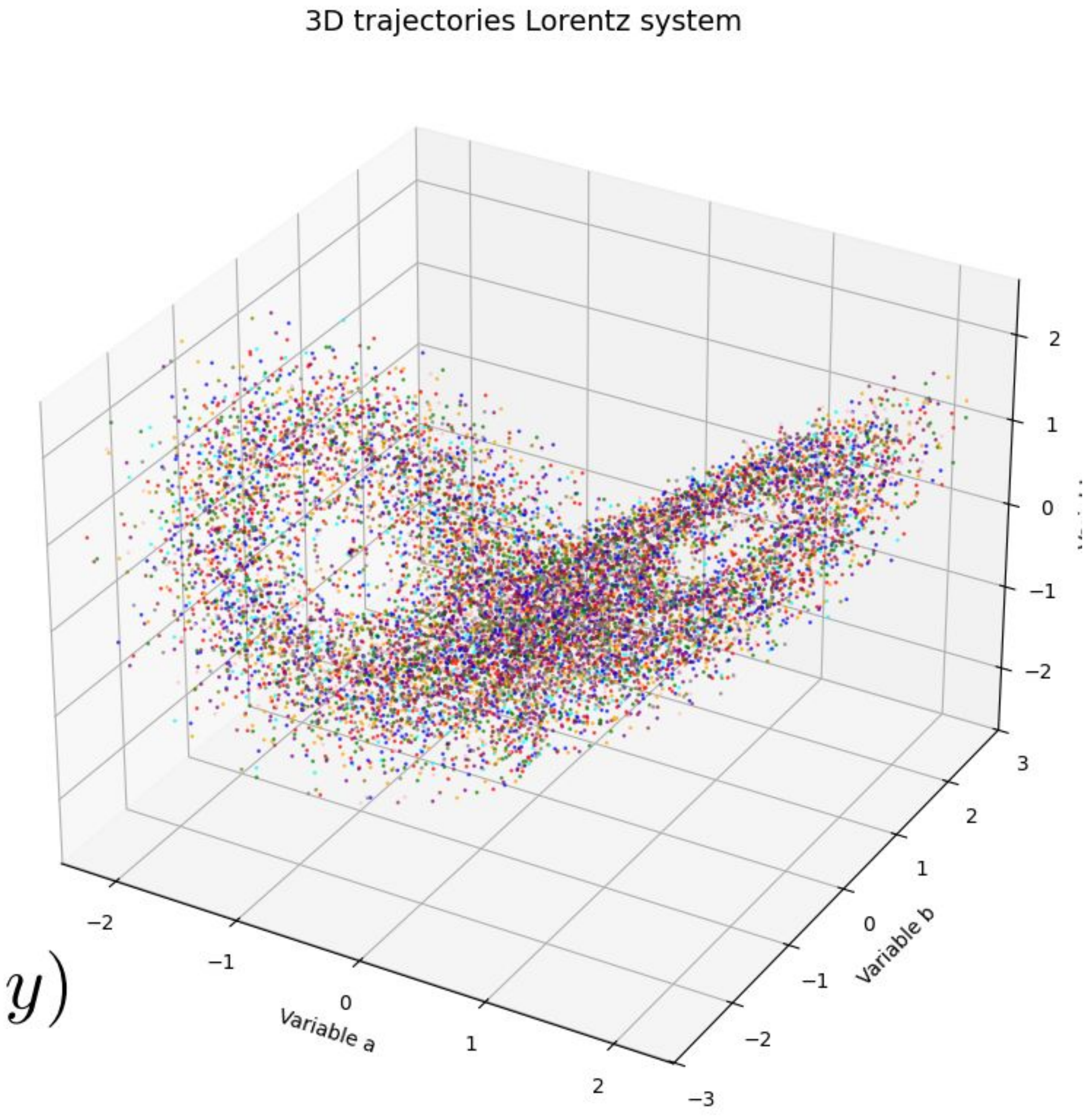
Goal - Retrieving the conditionnal distribution $p(x_{1:L} \mid y)$ of a simple stochastic dynamic model

Toy example: Lorentz 1963

- State x = (a, b, c)
- Evolving according to ODE

$$\dot{a} = \sigma(b - a)$$
$$\dot{b} = a(\rho - c) - b$$
$$\dot{c} = ab - \beta c$$

- We have an algorithm (BPF) to sample the groundtruth $p(x_{1:L} \mid y)$



Experimental setup

1 - Observations (y) generation

- ➢ Generate (1024x1024x3) dimensional states x
- ➢ Sample observations from two distinct processes (64 times)
  - ○ "lo" : low frequency, low variance $\mathcal{N}(y \mid \tilde{a}_{1:L:8}, 0.05^2 I)$
  - ○ "hi" : high frequency, high variance $\mathcal{N}(y \mid \tilde{a}_{1:L}, 0.25^2 I)$

Experimental setup

1 - Observations (y) generation

- ➢ Generate (1024x1024x3) dimensional states x
- ➢ Sample observations from two distinct processes (64 times)
  - ○ “lo” : low frequency, low variance $\mathcal{N}(y \mid \tilde{a}_{1:L:8}, 0.05^2 I)$
  - ○ “hi” : high frequency, high variance $\mathcal{N}(y \mid \tilde{a}_{1:L}, 0.25^2 I)$

2 - Building Groundtruth distributions $p(x_{1:L} \mid y)$

- ➢ For each observation we draw 1024 trajectories with BPF
- ➢ We obtain:
  - ○ x64 $p(x_{1:L} \mid y)$ drawn from “lo”
  - ○ x64 $p(x_{1:L} \mid y)$ drawn from “hi”

Experimental setup

1 - Observations (y) generation

- ➢ Generate (1024x1024x3) dimensional states x
- ➢ Sample observations from two distinct processes (64 times)
  - ○ "lo" : low frequency, low variance $\mathcal{N}(y \mid \tilde{a}_{1:L:8}, 0.05^2 I)$
  - ○ "hi" : high frequency, high variance $\mathcal{N}(y \mid \tilde{a}_{1:L}, 0.25^2 I)$

3 - Building SDA distributions $q(x_{1:L} \mid y)$

- ➢ For each observation we draw 1024 trajectories with SDA model
- ➢ We obtain:
  - ○ x64 $q(x_{1:L} \mid y)$ drawn from "lo"
  - ○ x64 $q(x_{1:L} \mid y)$ drawn from "hi"

Experimental setup

1 - Observations (y) generation

2 - Building Groundtruth distributions $p(x_{1:L} \mid y)$

3 - Building SDA distributions $q(x_{1:L} \mid y)$

4 - Comparison metrics

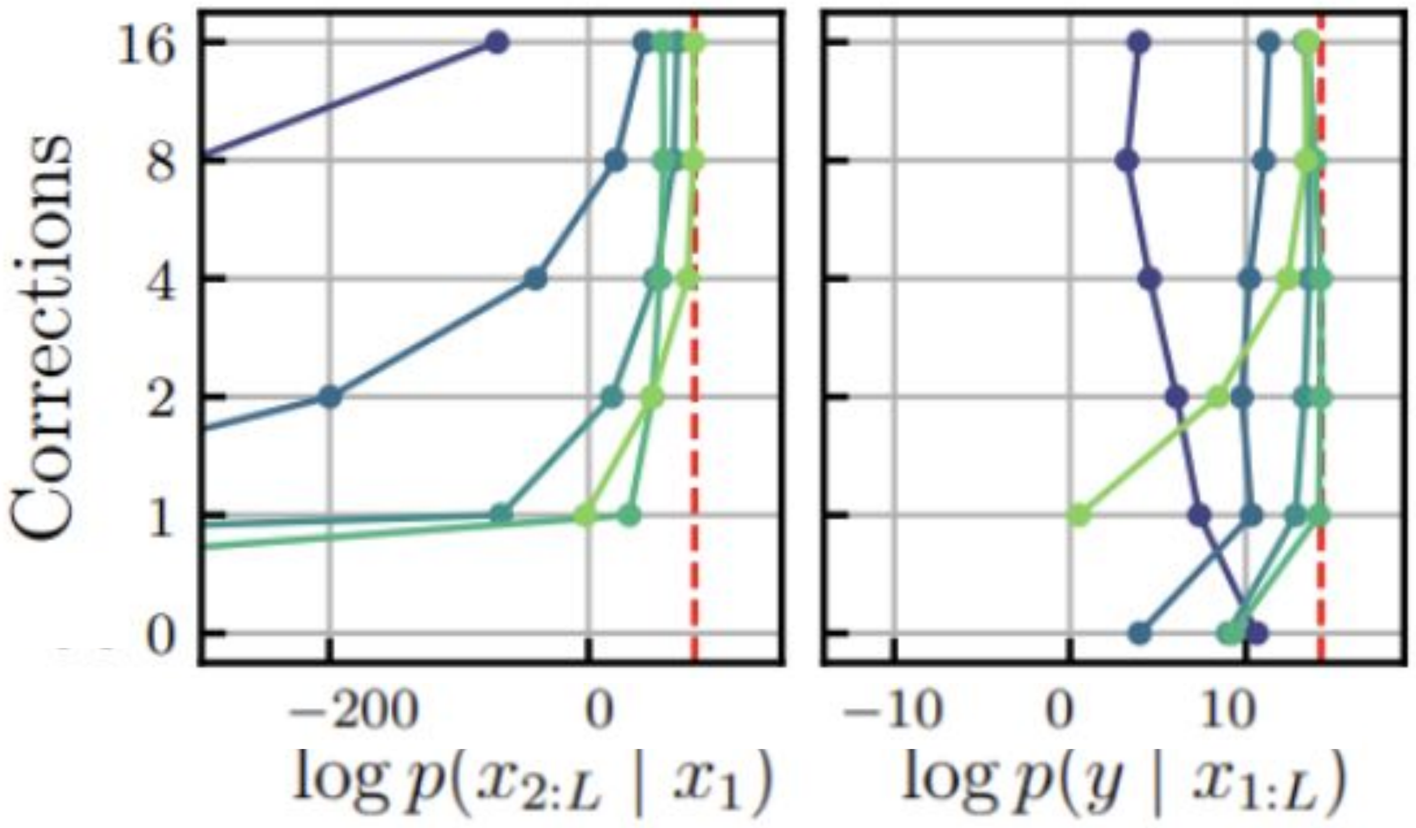
- ➢ Expected log-prior $\mathbb{E}_{q(x_{1:L} \mid y)}[\log p(x_{2:L} \mid x_1)]$
- ➢ Expected log likelihood $\mathbb{E}_{q(x_{1:L} \mid y)}[\log p(y \mid x_{1:L})]$

Comparison made with different values for k (markov window), C (correction steps) and observation types (“hi” or “lo”).

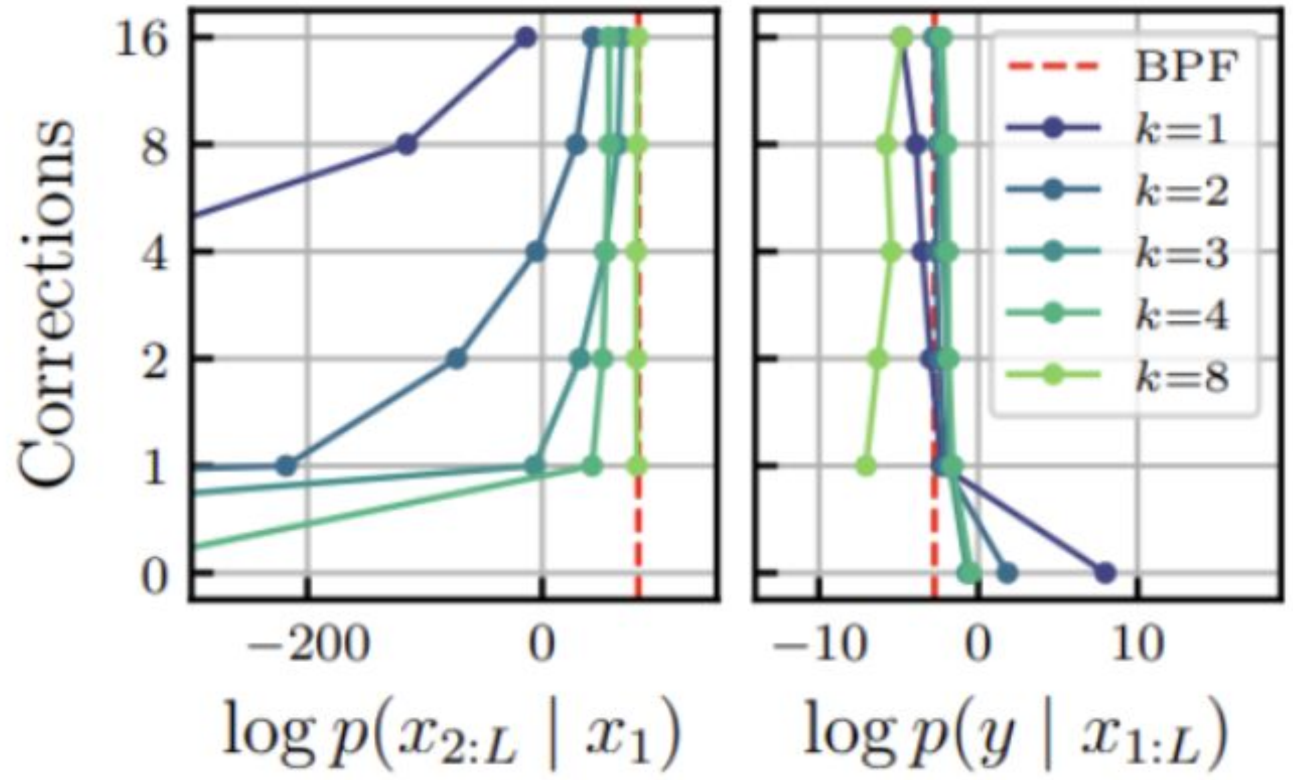
Results - Lorentz 1963

Comparison of SDA and groundtruth distributions (dotted lines)

“lo”



“hi”



- SDA distributions gets better by increasing C and k
- Posterior distributions for k > 3 and C > 2 are almost equivalent.

Conclusion

- Training on short segments, infer arbitrarily long trajectories — applicable beyond Markov chains to any structured set of variables

- New stable likelihood approximation directly transferable to inpainting, super-resolution, and scientific inverse problems

- Inference cost and scaling to operational systems remain open challenges

Next ?

Activity available on https://github.com/pdenailly/score-da-colab

GitHub

Thank you !